

Semantic HTML

COMPLETE LEARNING GUIDE

What You'll Learn

Semantic HTML means choosing elements based on what content means, not how it happens to look — an `<article>` instead of a styled `<div>`, a `<button>` instead of a clickable `<span>`. This matters far beyond style: the browser's accessibility tree, screen readers, search engine crawlers, and even browser features like 'reader mode' or automatic form autofill all depend on semantic structure to understand a page. A visually identical page built entirely from `<div>` and `<span>` tags is invisible in meaning to every one of those systems. This submodule covers the structural landmark elements that define a page's regions, the inline elements that convey meaning in text, how to build genuinely accessible forms, and how heading hierarchy and ARIA fill the remaining gaps. By the end you should be able to look at any UI mockup and know which real HTML element each piece should become before writing a single line of CSS.

1. Structural Landmarks: header, nav, main, article, section, aside, footer

HTML5 introduced a set of sectioning elements that describe the large-scale regions of a page instead of relying on generically-named `<div id="header">` conventions. `<header>` marks introductory content (often a page or section title plus navigation), `<nav>` wraps primary navigation links, `<main>` wraps the one primary content area unique to that page (there should only ever be one visible `<main>` per page), `<article>` wraps self-contained content that would make sense distributed on its own (a blog post, a product card, a comment), `<section>` groups thematically related content that usually has its own heading, `<aside>` holds tangentially related content (a sidebar, a pull quote), and `<footer>` holds closing metadata. Each of these creates an implicit ARIA landmark role, which is exactly what lets a screen reader user jump directly to 'navigation' or 'main content' via a shortcut instead of tabbing through the entire page linearly.

- `<main>` must be unique per page and should never be nested inside `<article>`, `<aside>`, `<header>`, footer, or another `<main>`.
- `<section>` is not a generic wrapper — if a region has no natural heading, it usually belongs in a `<div>` instead.
- `<article>` should make sense if pulled out of the page entirely and pasted somewhere else (an RSS reader, a syndication feed).

- Screen readers expose these as landmarks (role=banner, navigation, main, complementary, contentinfo) — inspect the accessibility tree in DevTools to see them.

HTML

```
<body>
  <header>
    <h1>Daily Times</h1>
    <nav aria-label="Primary">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/world">World</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <h2>Local elections draw record turnout</h2>
      <p>Published <time datetime="2026-09-14">Sept 14, 2026</time></p>
      <p>Turnout figures released this morning show...</p>
    </article>
    <aside aria-label="Related">
      <h2>Related coverage</h2>
      <ul><li><a href="/analysis">Full analysis</a></li></ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 Daily Times</p>
  </footer>
</body>
```

2. Text-Level Semantics: Meaning vs. Presentation

HTML distinguishes tags that convey meaning from tags that only convey visual style, and the two are not interchangeable even when they render identically by default. `<strong>` means 'this content has strong importance' and is announced with emphasis by screen readers; `<b>` means only 'stylistically offset text' with no implied importance. Likewise `<em>` means genuine emphasis (changes the meaning of a sentence if read aloud with stress), while `<i>` is for text that is set off in a different voice or mood — a technical term, a ship's name, a thought — without implying emphasis. Other useful semantic text elements include `<time>` (machine-readable dates/times via the `datetime` attribute), `<mark>` (highlighted/relevant text, such as a search match), `<abbr>` (an abbreviation with its expansion in the `title` attribute), and `<code>/<pre>` for literal code text.

- Use `<strong>/<em>` when meaning changes; use `<b>/<i>` only for pure styling with no semantic weight (rare in practice).

- `<time datetime="2026-09-18">Sept 18</time>` lets machines (calendars, search engines) parse the actual date regardless of the displayed text.
- `<abbr title="Cascading Style Sheets">CSS</abbr>` lets assistive tech announce the full expansion on first encounter.

3. Accessible Forms: Labels, Fieldsets, and Input Types

Forms are the highest-stakes place to get semantics right because a mislabeled field can make an entire form unusable with a screen reader or unreliable with password managers and autofill. Every input needs an associated `<label>`, connected either by wrapping the input inside the label or by matching a `for` attribute to the input's `id` — this also creates a larger, more accessible click target since clicking the label text focuses the input. Related fields (like a set of radio buttons for a single question, or a billing-address group) should be wrapped in `<fieldset>` with a `<legend>` describing the group. Using the most specific input type (email, tel, date, number, url) instead of a generic text input isn't cosmetic — it triggers the right mobile keyboard, enables built-in browser validation, and improves autofill accuracy.

HTML

```
<form>
  <div>
    <label for="email">Email address</label>
    <input id="email" name="email" type="email" required autocomplete="email" />
  </div>

  <fieldset>
    <legend>Preferred contact method</legend>
    <label><input type="radio" name="contact" value="email" /> Email</label>
    <label><input type="radio" name="contact" value="phone" /> Phone</label>
  </fieldset>

  <button type="submit">Create account</button>
</form>
```

4. Heading Hierarchy and the Document Outline

Headings (`<h1>` through `<h6>`) aren't just for making text bigger — screen reader users frequently navigate a page by pulling up a list of all headings and jumping directly to the section they need, the same way a sighted user visually scans a page for the right subheading. This only works if heading levels are used in order without skipping (an `<h2>` should not jump straight to an `<h4>` just because the smaller default font size looked better — that's a job for CSS, not for choosing a different heading level). Each page should generally have exactly one `<h1>` describing its main content, with nested sections using `<h2>`, then `<h3>` for subsections within those, and so on.

- Never choose a heading level for its default font size — style it with CSS and keep the level semantically correct.

- Run an automated heading-structure check (axe DevTools, Lighthouse accessibility audit, or the WAVE browser extension) as part of your normal review process.

5. ARIA: The Last Resort, Not the First Tool

The first rule of ARIA (Accessible Rich Internet Applications) is: don't use ARIA if a native HTML element already provides the semantics and behavior you need. A native `<button>` already has the correct role, is keyboard-focusable, responds to both Enter and Space, and is announced correctly by every screen reader with zero extra code. Recreating that with `<div role="button" tabindex="0">` requires you to manually wire up keyboard event handlers, manage focus styles, and set `aria-pressed` or similar state — and it's easy to miss an edge case native elements handle automatically. ARIA attributes are essential for the gaps HTML doesn't cover — custom widgets like tab panels, comboboxes, or live regions that announce dynamic updates (`aria-live`) — but they should be reached for only after confirming no semantic element already does the job.

- `aria-label` and `aria-labelledby` give an accessible name when visible text isn't enough or isn't present.
- `aria-live="polite"` announces dynamically inserted content (like a form validation error or a toast notification) without stealing focus.
- `role` overrides an element's default semantics — using it on an element that already has the right native role is redundant at best, harmful at worst.

6. Images, Alt Text, and Decorative Content

Every `<img>` needs an `alt` attribute, but writing good alt text is a skill: describe the content and function of the image as it relates to the surrounding context, not a literal visual description. An image that functions as a link to the homepage should have alt text describing the destination ('Acme homepage'), not the image itself ('logo'). Purely decorative images (a background flourish, a spacer) should use an empty `alt=""` (not omit the attribute entirely) so screen readers skip over them silently rather than announcing an unhelpful filename.

- `alt=""` (empty, but present) tells assistive tech to skip a purely decorative image.
- Omitting alt entirely is different from `alt=""` — some screen readers will read the filename as a fallback, which is worse than silence.
- Complex images (charts, diagrams) need a longer text alternative nearby in the page, since alt text alone is too short to convey the data.

Common Pitfalls

- Building entire layouts from `<div>` and `<span>` ('div soup') when a semantic element already exists for that purpose.
- Using a `<div>` or `<span>` with an `onClick` handler instead of a real `<button>`, which silently breaks keyboard navigation and screen reader announcement.

- Skipping heading levels for visual sizing reasons (jumping from <h2> straight to <h4>) instead of using CSS to control size.
- Leaving form inputs without an associated <label>, relying on placeholder text alone (which disappears once the user starts typing and isn't reliably announced).
- Omitting alt attributes on images, or filling them with unhelpful text like 'image123.jpg'.
- Reaching for ARIA roles and attributes before checking whether a native HTML element already provides the needed semantics and behavior.
- Using multiple <h1> elements or multiple <main> elements on a single page.

Key Takeaways

- Choose elements for what content means, not how it renders — CSS controls appearance, HTML controls meaning.
- Landmark elements (header/nav/main/aside/footer) let screen reader users jump directly between page regions instead of tabbing linearly.
- A native interactive element (button, a, input) gives you keyboard support and correct semantics for free — recreating it with a styled div means rebuilding all of that by hand.
- Every form input needs a real, programmatically associated label — placeholder text is not a substitute.
- The first rule of ARIA is not to use ARIA when a native HTML element already solves the problem.

Learn More

MDN — HTML: A good basis for accessibility

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Accessibility/HTML

web.dev — Learn HTML: Semantics

<https://web.dev/learn/html/semantic-html>

W3C WAI-ARIA Authoring Practices Guide

<https://www.w3.org/WAI/ARIA/apg/>